

MA725

Assignment 5 — Solving Nonlinear Systems

Individual Assignment — Maximum Marks: 30

Please take note that this assignment carries three times the weightage of a standard assignment, so we strongly encourage you to dedicate additional time and effort to it accordingly.

Problem Statement

The paper “*Newton’s Method for Solving Non-Linear Systems of Algebraic Equations (NLSAEs) with MATLAB/Simulink[®] and MAPLE[®]*” [1] demonstrates Newton’s Method for coupled nonlinear systems. In this assignment, you will **reproduce and extend** Equation (14) from that paper entirely in **Python**, compare Newton’s Method against Broyden’s Quasi-Newton method, and evaluate both methods using the R-square metric from Equation (55) of the paper.

Given System — Equation (14) from the Reference Paper

Solve the following 3×3 nonlinear system of algebraic equations:

$$\begin{cases} f_1(x_1, x_2, x_3) = 3x_1 - \cos(x_2x_3) - \frac{1}{2} = 0 \\ f_2(x_1, x_2, x_3) = x_1^2 - 81(x_2 + 0.1)^2 + \sin x_3 + 1.06 = 0 \\ f_3(x_1, x_2, x_3) = e^{-x_1x_2} + 20x_3 + \frac{10\pi - 3}{3} = 0 \end{cases} \quad (1)$$

Starting guess: $\mathbf{x}_0 = (0.1, 0.1, -0.1)$

Tolerance: $\varepsilon = 10^{-6}$

Max iterations: 50

Known solution from the reference paper: $x_1 \approx 0.5$, $x_2 \approx 0$, $x_3 \approx -0.5236 \approx -\pi/6$.

Evaluation Metric — R-square, Equation (55)

Use the R-square statistic to quantify convergence quality at each iteration. Given $SST = SSR + SSE$, the R-square is expressed as:

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST} \quad (2)$$

For this assignment, define: $SSE_k = \|\mathbf{F}(\mathbf{x}_k)\|_2^2$ (residual at iteration k) and $SST = \|\mathbf{F}(\mathbf{x}_0)\|_2^2$ (initial residual). Thus:

$$R_k^2 = 1 - \frac{\|\mathbf{F}(\mathbf{x}_k)\|_2^2}{\|\mathbf{F}(\mathbf{x}_0)\|_2^2}$$

As $R^2 \rightarrow 1$, the method converges. You will use this to compare Newton vs. Broyden.

Tasks

Task 1 Newton's Method from Scratch

[7 marks]

Implement Newton's Method **without** using `scipy.optimize` or any built-in root-finding solver.

- Derive the 3×3 analytical Jacobian $J(\mathbf{x})$ for the system in Equation (1) and hard-code it in your implementation.
- At each iteration k , solve the linear system

$$J(\mathbf{x}_k) \Delta \mathbf{x} = -\mathbf{F}(\mathbf{x}_k)$$

using `numpy.linalg.solve`, then update $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}$.

- Stop when $\|\mathbf{F}(\mathbf{x}_k)\|_2 < \varepsilon$.
- Print a full iteration table with columns: *Iteration* k , $\mathbf{x}_k$, $\|\mathbf{F}(\mathbf{x}_k)\|_2$.

Task 2 Broyden's Quasi-Newton Method

[7 marks]

Implement Broyden's Good Method, which approximates the Jacobian without recomputing it analytically at each step.

- Initialize $B_0 = J(\mathbf{x}_0)$ (the analytical Jacobian evaluated at the starting guess).
- At each iteration k , solve $B_k \Delta \mathbf{x} = -\mathbf{F}(\mathbf{x}_k)$, update $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}$, then apply the rank-1 Broyden update:

$$B_{k+1} = B_k + \frac{(\Delta \mathbf{f} - B_k \Delta \mathbf{x}) \Delta \mathbf{x}^\top}{\Delta \mathbf{x}^\top \Delta \mathbf{x}}$$

where $\Delta \mathbf{f} = \mathbf{F}(\mathbf{x}_{k+1}) - \mathbf{F}(\mathbf{x}_k)$ and $\Delta \mathbf{x} = \mathbf{x}_{k+1} - \mathbf{x}_k$.

- Use the same starting guess, tolerance, and max iterations as Task 1.
- Print the same iteration table format as Task 1.

Task 3 R-square Comparison Table & Plot

[8 marks]

- (a) Compute R_k^2 (Equation (2)) at every iteration for both methods.
- (b) Produce a **side-by-side comparison table** with the following columns:

Iteration k	$\ \mathbf{F}\ _2$ (Newton)	R^2 (Newton)	$\ \mathbf{F}\ _2$ (Broyden)	R^2 (Broyden)
0				
1				
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

- (c) Plot R^2 vs. iteration number for both methods on a single `matplotlib` figure. Label axes, include a legend, and save the figure as `convergence_rsquare.png`.
- (d) The table and plot must clearly **demonstrate the improvement from Newton's Method to Broyden's Method** in terms of convergence behaviour.

Deliverables

Submit the following files as a single `.zip` archive:

- A single `.py` file or Jupyter notebook (`.ipynb`) containing clean, commented code for Tasks 1–3.
- Printed iteration tables for both methods (terminal output or inside the notebook).
- The R-square side-by-side comparison table (printed or as a formatted notebook cell).
- The convergence plot saved as `convergence_rsquare.png`.
- Written analysis for Task 4 as a PDF or inside the notebook.

Constraints

Allowed libraries: `numpy`, `matplotlib`, `math`, `copy`

Not allowed: `scipy.optimize`, `sympy` auto-differentiation, or any built-in root-finding solver

Submissions that use prohibited libraries will receive zero marks for the affected task, regardless of correctness of the output.

Reference

- [1] Kisabo, A. B., Uchenna, N. C., & Adebimpe, F. A. (2017). Newton's method for solving non-linear system of algebraic equations (NLSAEs) with MATLAB/Simulink[®] and MAPLE[®]. *American Journal of Mathematical and Computer Modelling*, 2(4), 117–131.